

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib, and verify the setup by running a simple Reinforcement Learning program.

Program

```
import gymnasium as gym

# Create CartPole environment
env = gym.make("CartPole-v1")

state, info = env.reset()

print("Initial State:", state)

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    print(f"Step {i+1}: Reward = {reward}")

    if terminated or truncated:
        break

env.close()
```

Sample Output

```
----- RE.
Initial State: [-0.02879073  0.03541516 -0.04370134  0.02128738]
Step 1: Reward = 1.0
Step 2: Reward = 1.0
Step 3: Reward = 1.0
Step 4: Reward = 1.0
Step 5: Reward = 1.0
>>> |
```

Experiment 2: Familiarization with OpenAI Gym/Gymnasium

Environments

Aim

To create, initialize, and interact with standard Reinforcement Learning environments such as **FrozenLake**, **CartPole**, and **MountainCar** using **Gymnasium** to understand the concepts of **states**, **actions**, **rewards**, and **episodes**.

Requirements

- Python 3.10+
- Gymnasium
- NumPy

Install Gymnasium if not already installed:

```
pip install gymnasium
```

Theory

Gymnasium is a Reinforcement Learning library that provides a collection of benchmark environments for developing and testing RL algorithms.

Key Concepts

- **State:** Current condition of the environment.
- **Action:** Decision taken by the agent.
- **Reward:** Feedback received after performing an action.
- **Episode:** A complete sequence from environment reset until termination.

Sample Output

```
===== RESTART: C:/Users/harik/Downloads/Program 2-2-CartPole Env
Choose Environment
1. FrozenLake
2. CartPole
3. MountainCar
Enter your choice (1-3): 2

Initial State: [-0.00869202 -0.01161155  0.03666633  0.0379526 ]

Step 1
Action: 1
State: [-0.00892425  0.18296595  0.03742538 -0.24293998]
Reward: 1.0

Step 2
Action: 0
State: [-0.00526493 -0.01267004  0.03256658  0.06130901]
Reward: 1.0

Step 3
Action: 0
State: [-0.00551833 -0.20824343  0.03379276  0.36408642]
Reward: 1.0

Step 4
Action: 1
State: [-0.0096832  -0.01361763  0.04107449  0.08224758]
Reward: 1.0

Step 5
Action: 1
State: [-0.00995556  0.18089218  0.04271944 -0.19719855]
Reward: 1.0
>>>
===== OpenAI Gymnasium Environments =====
1. FrozenLake
2. CartPole
3. MountainCar
4. Exit
Enter your choice: 1

Selected Environment: FrozenLake
Initial State: 0

Step 1
Action : 3
State  : 0
Reward : 0

Step 2
Action : 3
State  : 0
Reward : 0

Step 3
Action : 0
State  : 0
Reward : 0

Step 4
Action : 2
State  : 1
Reward : 0

Step 5
Action : 1
State  : 5
Reward : 0
Episode Finished
```

```

===== OpenAI Gymnasium Environments =====
1. FrozenLake
2. CartPole
3. MountainCar
4. Exit
Enter your choice: 2

Selected Environment: CartPole
Initial State: [-0.02748351  0.03079834  0.01174735  0.0225543 ]

Step 1
Action : 0
State  : [-0.02686755 -0.16449009  0.01219844  0.31892037]
Reward : 1.0

Step 2
Action : 0
State  : [-0.03015735 -0.35978362  0.01857685  0.61542517]
Reward : 1.0

Step 3
Action : 1
State  : [-0.03735302 -0.1649261   0.03088535  0.32865056]
Reward : 1.0

Step 4
Action : 1
State  : [-0.04065154  0.02974287  0.03745836  0.04586522]
Reward : 1.0

Step 5
Action : 1
State  : [-0.04005668  0.22430822  0.03837566 -0.2347679 ]
Reward : 1.0

===== OpenAI Gymnasium Environments =====
1. FrozenLake
2. CartPole
3. MountainCar
4. Exit
Enter your choice: 4
Program Exited.
>

```

Experiment 3: Implementation of the Reinforcement Learning Framework Aim

To implement a basic Reinforcement Learning (RL) framework where an agent interacts with an environment using states, actions, rewards, and policies.

Program

```

import gymnasium as gym

env = gym.make("CartPole-v1")

state, info = env.reset()
total_reward = 0

for step in range(10):
    action = env.action_space.sample() # Random action
    state, reward, terminated, truncated, info = env.step(action)

    print(f"Step {step+1}: Action={action}, Reward={reward}")

    total_reward += reward

    if terminated or truncated:
        break

print("Total Reward:", total_reward)

env.close()

```

Sample Output

```
>>
=====
Step 1: Action=1, Reward=1.0
Step 2: Action=0, Reward=1.0
Step 3: Action=1, Reward=1.0
Step 4: Action=0, Reward=1.0
Step 5: Action=0, Reward=1.0
Step 6: Action=0, Reward=1.0
Step 7: Action=0, Reward=1.0
Step 8: Action=0, Reward=1.0
Step 9: Action=1, Reward=1.0
Step 10: Action=0, Reward=1.0
Total Reward: 10.0
>> |
```

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To simulate a simple Markov Decision Process (MDP) demonstrating state transitions, rewards, and policy execution.

Program

```
# MDP Simulation
```

```
states = ["A", "B", "Goal"]
```

```
transitions = {
    "A": "B",
    "B": "Goal"
}
```

```
rewards = {
    "A": 0,
    "B": 5,
    "Goal": 10
}
```

```
state = input("Enter the starting state (A/B): ")
```

```
if state not in transitions:
    print("Invalid starting state!")
```

```
else:
    total_reward = 0
```

```
    while state != "Goal":
```

```

print("\nCurrent State:", state)

reward = rewards[state]
total_reward += reward

print("Reward Received:", reward)

state = transitions[state]

print("\nCurrent State:", state)
print("Reward Received:", rewards[state])

total_reward += rewards[state]

print("\nTotal Reward:", total_reward)

```

Sample Output

```

=====
State: A
State: B
State: Goal
Total Reward: 11
|
'''
=====
Enter the starting state (A/B): A

Current State: A
Reward Received: 0

Current State: B
Reward Received: 5

Current State: Goal
Reward Received: 10

Total Reward: 15
>>>
=====
Enter the starting state (A/B): B

Current State: B
Reward Received: 5

Current State: Goal
Reward Received: 10

Total Reward: 15
>>>

```

Experiment 5: Bellman Equation Demonstration

Aim

To implement the Bellman Equation for estimating state values.

Program

Bellman Equation Demonstration

```

gamma = float(input("Enter discount factor (0 to 1): "))
reward = int(input("Enter immediate reward: "))
future_value = float(input("Enter future state value: "))

state_value = reward + gamma * future_value

print("\nUsing Bellman Equation")
print("State Value =", state_value)

```

Sample Output

```
=====
Enter discount factor (0 to 1): 0.002
Enter immediate reward: 3
Enter future state value: 6

Using Bellman Equation
State Value = 3.012
>>
```

Experiment 6: Exploration vs Exploitation

Aim

To compare Greedy, ϵ -Greedy, and Random action-selection strategies.

Program

```
import random

actions = ["Left", "Right"]

epsilon = 0.2

if random.random() < epsilon:
    action = random.choice(actions)
else:
    action = "Right"

print("Selected Action:", action)
```

Sample Output

```
=====
Enter epsilon value (0 to 1): 0.4

Strategy Used : Exploitation
Selected Action: Right
>>
```

Experiment 7: Multi-Armed Bandit Problem

Aim

To implement the ϵ -Greedy algorithm for the Multi-Armed Bandit problem.

Program

```
import random

rewards = [1, 2, 5]

for i in range(5):
    arm = random.randint(0, 2)
    print("Arm:", arm, "Reward:", rewards[arm])
```

Sample Output

```

=====
Arm: 0 Reward: 1
Arm: 0 Reward: 1
Arm: 2 Reward: 5
Arm: 2 Reward: 5
Arm: 2 Reward: 5
>>>

```

Experiment 8: Reward Visualization

Aim

To visualize cumulative rewards using Matplotlib.

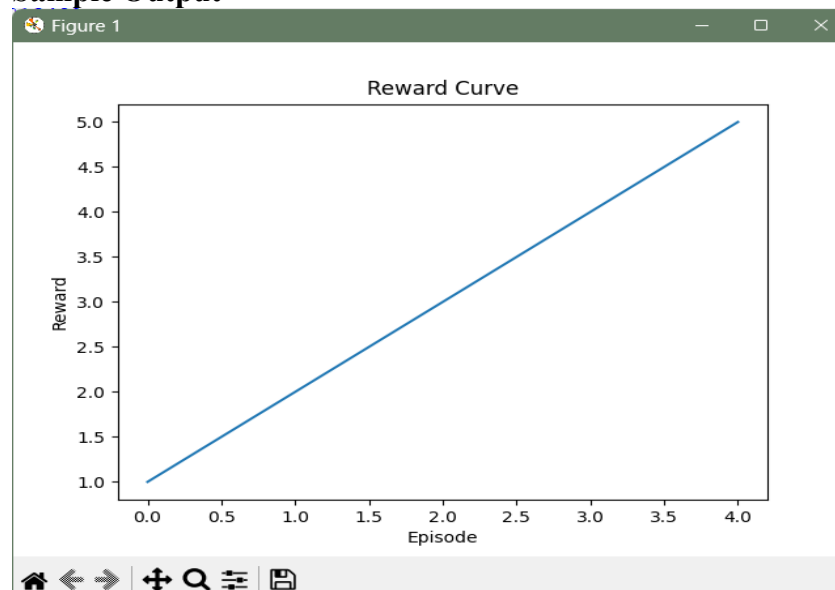
Program

```
import matplotlib.pyplot as plt
```

```
rewards = [1, 2, 3, 4, 5]
```

```
plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Reward Curve")
plt.show()
```

Sample Output



Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim

To build a simple neural network using TensorFlow and Keras.

Program

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential([
    Dense(16, activation="relu", input_shape=(4,)),
    Dense(2, activation="linear")
])
```

])

model.summary()

Sample Output

```
Model: "sequential"

```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	80
dense_1 (Dense)	(None, 2)	34

```
Total params: 114 (456.00 B)
Trainable params: 114 (456.00 B)
Non-trainable params: 0 (0.00 B)
>
```

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim

To simulate an RL agent in the CartPole environment.

Program

```
import gymnasium as gym
```

```
env = gym.make("CartPole-v1")
```

```
state, info = env.reset()
```

```
for i in range(10):
```

```
    action = env.action_space.sample()
```

```
    state, reward, terminated, truncated, info = env.step(action)
```

```
    print(f"Step {i+1}: Reward={reward}")
```

```
    if terminated or truncated:
```

```
        break
```

```
env.close()
```

Sample Output

```
=====
Step 1: Reward=1.0
Step 2: Reward=1.0
Step 3: Reward=1.0
Step 4: Reward=1.0
Step 5: Reward=1.0
Step 6: Reward=1.0
Step 7: Reward=1.0
Step 8: Reward=1.0
Step 9: Reward=1.0
Step 10: Reward=1.0
>>> |
```